```
# Developer quickstart

import {
  Assistant,
  Camera,
  ChatTripleDots,
  Code,
  Bolt,
  Speed,
  SquarePlus,
} from "@components/react/oai/platform/ui/Icon.react";
```

The OpenAI API provides a simple interface to state-of-the-art AI [models](https://developers.openai.com/api/docs/models) for text generation, natural language processing, computer vision, and more. Get started by creating an API Key and running your first API call. Discover how to generate text, analyze images, build agents, and more.

Create and export an API key

StatsigClient.logEvent("quickstart_create_api_key_click", null, null)

}

>

Create an API Key

Before you begin, create an API key in the dashboard, which you'll use to

securely [access the API](https://developers.openai.com/api/docs/api-reference/authentication). Store the key

in a safe location, like a [`.zshrc`

file](https://www.freecodecamp.org/news/how-do-zsh-configuration-files-work/) or

another text file on your computer. Once you've generated an API key, export it

as an [environment variable](https://en.wikipedia.org/wiki/Environment_variable)

in your terminal.

<div data-content-switcher-pane data-value="macOS">

macOS / Linux

Export an environment variable on macOS or Linux systems

```bash
export OPENAI_API_KEY="your_api_key_here"
```

<div data-content-switcher-pane data-value="windows" hidden>
Windows
Export an environment variable in PowerShell

```bash
setx OPENAI_API_KEY "your_api_key_here"
```

OpenAI SDKs are configured to automatically read your API key from the system environment.

Install the OpenAI SDK and Run an API Call

<div data-content-switcher-pane data-value="javascript">
JavaScript

<div data-content-switcher-pane data-value="python" hidden>
Python

```
    </div>
  <div data-content-switcher-pane data-value="csharp" hidden>
    <div class="hidden">.NET</div>
    </div>
  <div data-content-switcher-pane data-value="java" hidden>
    <div class="hidden">Java</div>
    </div>
  <div data-content-switcher-pane data-value="golang" hidden>
    <div class="hidden">Go</div>
    </div>


<a
  href="https://github.com/openai/openai-responses-starter-app"
  target="_blank"
  rel="noreferrer"
>


<span slot="icon">
      </span>
    Start building with the Responses API.


</a>

[

<span slot="icon">
```

```
    </span>
   Learn more about prompting, message roles, and building conversational apps.

](https://developers.openai.com/api/docs/guides/text)

## Add credits to keep building




StatsigClient.logEvent("quickstart_add_credits_billing_click", null, null)
  }
>
  Go to billing



{/* prettier-ignore */}
<div className="mt-2">Congrats on running a free test API request! Start building real applications with higher limits and use <a href="/api/docs/models" target="_blank">our models</a> to generate text, audio, images, videos and more.</div>

<div className="mt-2">
  Access dashboard features designed to help you ship faster:
</div>
<a
  href="https://platform.openai.com/chat"
  target="_blank"
  rel="noreferrer"
  onClick={() =>
    StatsigClient.logEvent(
```

```
      "quickstart_add_credits_chat_playground_click",
      null,
      null
    )
  }
>



<span slot="icon">
      </span>
    Build & test conversational prompts and embed them in your app.



</a>
<a
  href="https://platform.openai.com/agent-builder"
  target="_blank"
  rel="noreferrer"
  onClick={() =>
    StatsigClient.logEvent(
      "quickstart_add_credits_agent_builder_click",
      null,
      null
    )
  }
>



<span slot="icon">
```

Build, deploy, and optimize agent workflows.

</a>

Analyze images and files

Send image URLs, uploaded files, or PDF documents directly to the model to extract text, classify content, or detect visual elements.

<div data-content-switcher-pane data-value="image-url">

Image URL

<div data-content-switcher-pane data-value="file-url" hidden>

File URL

Use a file URL as input

```bash
curl "https://api.openai.com/v1/responses" \\
  -H "Content-Type: application/json" \\
  -H "Authorization: Bearer $OPENAI_API_KEY" \\
  -d '{
    "model": "gpt-5",
    "input": [
      {
        "role": "user",
```

```
            "content": [
              {
                "type": "input_text",
                "text": "Analyze the letter and provide a summary of the key points."
              },
              {
                "type": "input_file",
                "file_url": "https://www.berkshirehathaway.com/letters/2024ltr.pdf"
              }
            ]
          }
        ]
    }'
```

```javascript
import OpenAI from "openai";
const client = new OpenAI();

const response = await client.responses.create({
    model: "gpt-5",
    input: [
        {
            role: "user",
            content: [
                {
                    type: "input_text",
                    text: "Analyze the letter and provide a summary of the key points.",
                },
```

```
            {
                type: "input_file",
                file_url: "https://www.berkshirehathaway.com/letters/2024ltr.pdf",
            },
        ],
    },
  ],
});

console.log(response.output_text);
```

```python
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
  model="gpt-5",
  input=[
    {
      "role": "user",
      "content": [
        {
          "type": "input_text",
          "text": "Analyze the letter and provide a summary of the key points.",
        },
        {
          "type": "input_file",
          "file_url": "https://www.berkshirehathaway.com/letters/2024ltr.pdf",
```

```
            },
          ],
        },
    ]
)

print(response.output_text)
```

```csharp
using OpenAI.Files;
using OpenAI.Responses;

string key = Environment.GetEnvironmentVariable("OPENAI_API_KEY")!;
OpenAIResponseClient client = new(model: "gpt-5", apiKey: key);

using HttpClient http = new();
using Stream stream = await http.GetStreamAsync("https://www.berkshirehathaway.com/letters/2024ltr.pdf");
OpenAIFileClient files = new(key);
OpenAIFile file = files.UploadFile(stream, "2024ltr.pdf", FileUploadPurpose.UserData);

OpenAIResponse response = (OpenAIResponse)client.CreateResponse([
    ResponseItem.CreateUserMessageItem([
        ResponseContentPart.CreateInputTextPart("Analyze the letter and provide a summary of the key points."),
        ResponseContentPart.CreateInputFilePart(file.Id),
    ]),
]);
```

```
Console.WriteLine(response.GetOutputText());
```

<div data-content-switcher-pane data-value="file-upload" hidden>

Upload file

Upload a file and use it as input

```bash
curl https://api.openai.com/v1/files \\
  -H "Authorization: Bearer $OPENAI_API_KEY" \\
  -F purpose="user_data" \\
  -F file="@draconomicon.pdf"

curl "https://api.openai.com/v1/responses" \\
  -H "Content-Type: application/json" \\
  -H "Authorization: Bearer $OPENAI_API_KEY" \\
  -d '{
    "model": "gpt-5",
    "input": [
      {
        "role": "user",
        "content": [
          {
            "type": "input_file",
            "file_id": "file-6F2ksmvXxt4VdoqmHRw6kL"
          },
          {
```

```
                "type": "input_text",
                "text": "What is the first dragon in the book?"
            }
        ]
    }
  ]
}'
```

```javascript
import fs from "fs";
import OpenAI from "openai";
const client = new OpenAI();

const file = await client.files.create({
  file: fs.createReadStream("draconomicon.pdf"),
  purpose: "user_data",
});

const response = await client.responses.create({
  model: "gpt-5",
  input: [
    {
      role: "user",
      content: [
        {
          type: "input_file",
          file_id: file.id,
        },
```

```
            {
                type: "input_text",
                text: "What is the first dragon in the book?",
            },
        ],
    },
  ],
});

console.log(response.output_text);
```

```python
from openai import OpenAI
client = OpenAI()

file = client.files.create(
  file=open("draconomicon.pdf", "rb"),
  purpose="user_data"
)

response = client.responses.create(
  model="gpt-5",
  input=[
    {
      "role": "user",
      "content": [
        {
          "type": "input_file",
```

```
                "file_id": file.id,
            },
            {
                "type": "input_text",
                "text": "What is the first dragon in the book?",
            },
        ]
    }
  ]
)

print(response.output_text)
```

```csharp
using OpenAI.Files;
using OpenAI.Responses;

string key = Environment.GetEnvironmentVariable("OPENAI_API_KEY")!;
OpenAIResponseClient client = new(model: "gpt-5", apiKey: key);

OpenAIFileClient files = new(key);
OpenAIFile file = files.UploadFile("draconomicon.pdf", FileUploadPurpose.UserData);

OpenAIResponse response = (OpenAIResponse)client.CreateResponse([
  ResponseItem.CreateUserMessageItem([
    ResponseContentPart.CreateInputFilePart(file.Id),
    ResponseContentPart.CreateInputTextPart("What is the first dragon in the book?"),
  ]),
```

```
]);

Console.WriteLine(response.GetOutputText());
```

[

Learn to use image inputs to the model and extract meaning from images.

](https://developers.openai.com/api/docs/guides/images)

[

Learn to use file inputs to the model and extract meaning from documents.

](https://developers.openai.com/api/docs/guides/file-inputs)

Extend the model with tools

Give the model access to external data and functions by attaching [tools](https://developers.openai.com/api/docs/guides/tools). Use built-in tools like web search or file search, or define your own for calling APIs, running code, or integrating with third-party systems.

<div data-content-switcher-pane data-value="web-search">

Web search

<div data-content-switcher-pane data-value="file-search" hidden>

File search

Search your files in a response

```python
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-4.1",
    input="What is deep research by OpenAI?",
    tools=[{
        "type": "file_search",
        "vector_store_ids": ["<vector_store_id>"]
    }]
)
print(response)
```

```javascript
```

```
import OpenAI from "openai";
const openai = new OpenAI();

const response = await openai.responses.create({
  model: "gpt-4.1",
  input: "What is deep research by OpenAI?",
  tools: [
    {
      type: "file_search",
      vector_store_ids: ["<vector_store_id>"],
    },
  ],
});
console.log(response);
```

```csharp
using OpenAI.Responses;

string key = Environment.GetEnvironmentVariable("OPENAI_API_KEY")!;
OpenAIResponseClient client = new(model: "gpt-5", apiKey: key);

ResponseCreationOptions options = new();
options.Tools.Add(ResponseTool.CreateFileSearchTool(["<vector_store_id>"]));

OpenAIResponse response = (OpenAIResponse)client.CreateResponse([
  ResponseItem.CreateUserMessageItem([
    ResponseContentPart.CreateInputTextPart("What is deep research by OpenAI?"),
  ]),
```

```
], options);

Console.WriteLine(response.GetOutputText());
```

```
  </div>
  <div data-content-switcher-pane data-value="function-calling" hidden>
    <div class="hidden">Function calling</div>
    </div>
  <div data-content-switcher-pane data-value="remote-mcp" hidden>
    <div class="hidden">Remote MCP</div>
    Call a remote MCP server
```

```bash
curl https://api.openai.com/v1/responses \\
-H "Content-Type: application/json" \\
-H "Authorization: Bearer $OPENAI_API_KEY" \\
-d '{
  "model": "gpt-5",
    "tools": [
      {
        "type": "mcp",
        "server_label": "dmcp",
        "server_description": "A Dungeons and Dragons MCP server to assist with dice rolling.",
        "server_url": "https://dmcp-server.deno.dev/sse",
        "require_approval": "never"
      }
    ],
    "input": "Roll 2d4+1"
```

```
  }'
```

```javascript
import OpenAI from "openai";
const client = new OpenAI();

const resp = await client.responses.create({
  model: "gpt-5",
  tools: [
    {
      type: "mcp",
      server_label: "dmcp",
      server_description: "A Dungeons and Dragons MCP server to assist with dice rolling.",
      server_url: "https://dmcp-server.deno.dev/sse",
      require_approval: "never",
    },
  ],
  input: "Roll 2d4+1",
});

console.log(resp.output_text);
```

```python
from openai import OpenAI

client = OpenAI()
```

```
resp = client.responses.create(
    model="gpt-5",
    tools=[
        {
            "type": "mcp",
            "server_label": "dmcp",
            "server_description": "A Dungeons and Dragons MCP server to assist with dice rolling.",
            "server_url": "https://dmcp-server.deno.dev/sse",
            "require_approval": "never",
        },
    ],
    input="Roll 2d4+1",
)

print(resp.output_text)
```

```csharp
using OpenAI.Responses;

string key = Environment.GetEnvironmentVariable("OPENAI_API_KEY")!;
OpenAIResponseClient client = new(model: "gpt-5", apiKey: key);

ResponseCreationOptions options = new();
options.Tools.Add(ResponseTool.CreateMcpTool(
    serverLabel: "dmcp",
    serverUri: new Uri("https://dmcp-server.deno.dev/sse"),
    toolCallApprovalPolicy: new
McpToolCallApprovalPolicy(GlobalMcpToolCallApprovalPolicy.NeverRequireApproval)
```

```
));

OpenAIResponse response = (OpenAIResponse)client.CreateResponse([
    ResponseItem.CreateUserMessageItem([
        ResponseContentPart.CreateInputTextPart("Roll 2d4+1")
    ])
], options);

Console.WriteLine(response.GetOutputText());
```

[

Learn about powerful built-in tools like web search and file search.

](https://developers.openai.com/api/docs/guides/tools)

[

Learn to enable the model to call your own custom code.

](https://developers.openai.com/api/docs/guides/function-calling)

Stream responses and build realtime apps

Use server-sent [streaming events](https://developers.openai.com/api/docs/guides/streaming-responses) to show results as they're generated, or the [Realtime API](https://developers.openai.com/api/docs/guides/realtime) for interactive voice and multimodal apps.

[

Use server-sent events to stream model responses to users fast.

](https://developers.openai.com/api/docs/guides/streaming-responses)

[

Use WebRTC or WebSockets for super fast speech-to-speech AI apps.

](https://developers.openai.com/api/docs/guides/realtime)

Build agents

Use the OpenAI platform to build [agents](https://developers.openai.com/api/docs/guides/agents) capable of taking action—like [controlling computers](https://developers.openai.com/api/docs/guides/tools-computer-use)—on behalf of your

users. Use the Agents SDK for [Python](https://openai.github.io/openai-agents-python) or [TypeScript](https://openai.github.io/openai-agents-js) to create orchestration logic on the backend.

[

Learn how to use the OpenAI platform to build powerful, capable AI agents.

](https://developers.openai.com/api/docs/guides/agents)